

Eye of Noruas

By Jacob Marcotte (Tamorcet) and Steven Hubbard (piratboy1)

Introduction

Our project's purpose was to create a program that could analyze network traffic and send alerts to a user if malware was discovered. The digital forensics problem that was solved was the identification and analysis of malware as it moves throughout a network.

Technical Implementation

The Eye of Noruas exists as a modified packet sniffer. When installed on a system, it monitors all TCP and UDP ports and scans for traffic entering the system. The file is compared to a database of patterns associated with malicious files. When a malicious file is located, AI is used to analyze the file and output its attributes and what actions should be taken to mitigate the damage that could be caused by malware.

The first problem we faced during our project was being able to compare large files within a network with large files stored in our database. Jacob concluded that the files could be more effectively compared and recognized if hashes were taken of them instead. A hash is a hexadecimal set of characters that can be taken of a file using an algorithm. The chances of two files sharing the same hash value is nearly impossible, making it an idea method for identifying a malware file. Steven suggested that the hashing algorithm SHA256 should be used due to its long length and complexity. Instead of comparing an entire file with a database, only 64 characters would need to be compared instead.

Once a hashing algorithm was agreed upon, Jacob began researching online databases containing hashes of known malicious data. The original plan for this project was for individual packets of network data to be analyzed instead of entire files. However, there were no publicly available databases of packet hashes on the internet, which prompted us to switch over from scanning packet hashes to scanning file hashes. The database that Jacob decided should be used in our project was a GitHub repository named Flagged_Hash_List made by a user named LGOG (https://github.com/LGOG/Flagged_Hash_list). The ReadMe file stated that this database was public, and that anyone with an interest in Cybersecurity may use it freely. Once the database was downloaded, Jacob filtered all non SHA256 hashes from the database so it could be used exclusively for our program.

Steven began writing the code for our project using Python. Steven already had a good portion of the code written from another one of his projects, which he then modified to be usable in our current project. Since Jacob was less experienced with Python (and more experienced with Java), we agreed that Steven should focus on the code while Jacob focused on everything else.

Once the code began being written, Jacob reached out to our professor to gain access to an AI server that could be used to support the project. Since ISU had disabled Sushi, a server that ISU students could utilize to incorporate AI into their projects, a new form of software was required. In response to this, Professor Sean Sanders decided to give us access to his personal web server to use AI instead. However, Steven was able to incorporate AI into our project without Sanders's web server, so that resource became irrelevant. The AI that we decided to use for our project was Chat GPT due to its exceptionally large database, and it was implemented without assistance.

Results

The following results of our project were taken from screenshots of our video presentation.

```
=====
MALICIOUS HASH DETECTED
=====
src_ip: 127.0.0.1
dst_ip: 127.0.0.1
src_port: 46042
dst_port: 5555
protocol: TCP
match_type: packet_payload
sha256: b30f69deb82ab77e90584ee7c6524332bcb73112e0aa32d1e393b2be88d6d3bc
size: 15
timestamp: 2026-05-07 12:32:35
description: Matched SHA-256 hash from packet payload.

-----
RECOMMENDED FIXES
-----
RECOMMENDED FIXES
-----
1) What likely happened
- The traffic is **loopback-to-loopback (127.0.0.1 → 127.0.0.1) over TCP**, so this was **local inter-process communication** on the same host, not external network activity.
- A local process connected from an **ephemeral client port (46042)** to a **listening service on port 5555**.
- The detector matched a **known-malicious SHA-256** in a **reassembled TCP stream buffer** ('tcp_stream_buffer'), meaning the signature was found in the **payload after stream reassembly** (not just a single packet).
- The **payload size is only 15 bytes**, suggesting a small token/command/handshake fragment rather than a full file transfer. This often aligns with **malware beacons/commands**, **local agent control ports**, or **a malicious local proxy**.*.

2) What system/service to check (port/protocol specific)
- **Identify what is listening on TCP/5555** on that Ubuntu host and which process initiated the client side.
- Common legitimate uses of 5555 include **Android ADB** (often 5555), custom dev services, or test APIs-however, on a server it can also be a **malware control port**.
- Because it's loopback, focus on:
  - The **process bound to 127.0.0.1:5555**
```

For this result, Steven created a custom file that could be detected by our program. We added the hash of the file to our database, allowing it to be detected. When detected, an analysis of the file containing metadata and a description of how to respond to it were printed in the terminal. I suggested creating an output file, but time constraints left us with too little time to implement that feature.

Lessons Learned and Conclusion

Before discussing the conclusion, we thought we should discuss both of our roles in this project. Both of us contributed towards this project in our own ways, and we feel as though we did an equal amount of work (contrary to what the commit logs say on our GitHub repository). We have both agreed to write each other's summary in this section.

Steven Hubbard (by Jacob)

I am thankful that I was able to participate in a project with him. Without him, I do not think our project would have turned out the way it did. His knowledge of Python was what allowed us to implement our ideas and create something we're proud of. The fact that he was able to find time to work on this despite his schedule is nothing short of spectacular. He wrote ALL of the code for our project (a lot of it was left over from one of his older projects), and I will never be able to thank him enough for his work. I know we said that we both agreed we did an equal amount of work, but I would argue that he did almost twice as much as me.

Jacob Marcotte (by Steven)

I know Jacob has exaggerated my role in this project, but he needs to give himself some credit where credit is due. While I handled the big stuff like the code, he handled a dozen other smaller things that made my life easier. He gave our professor updates on our project while asking for resources and help when I wasn't around, he found a database of malware hashes and filtered them, and he even went out of his way to offer me help whenever I needed it. He couldn't do much with Python, but he never stopped asking. The whole idea of how this project works was his. He created the name, he designed the structure (how hashes and AI would be implemented) and even outlined the format of our video presentation. He's the reason this project exists at all! If he learned python, he could probably do it himself. I'm glad we were able to work together on this.

The most valuable lessons learned from this project were the importance of communication and scheduling. The reason we took so long to begin our project was because we were awaiting resources from ISU to be used to work on our project. For communication, we were not aware that we needed to speak with our professor about obtaining VMs from ISU, and assumed that our list of requirements we sent him in our project proposal would be enough. When it came to scheduling, we found it difficult to find times that worked between the two of us. Because Steven is a member of the US military, he would often be unavailable for several

days to participate in military exercises and drills. Scheduling became a recurring issue when it came to teamwork.

Another lesson we learned from this project was the importance of having different perspectives on different topics. Jacob's knowledge of resource management and Steven's knowledge of Python allowed the project to evolve in a way which allowed it to be completed on time. Our different skillsets allowed us both to fill in the blanks in areas which the other person was unfamiliar with. Having a team with diverse skillsets is essential to creating a product that functions properly. In the time that we have been attending ISU, we have recognized that working alongside people with too many similar interests and knowledge bases can cause a project to stagnate, so being able to experience this project the way we have has allowed us to gain insight into how an effective team of programmers should act.

Finally (we are adding this segment reluctantly), we both learned more about AI implementation and how it can be added to enhance software. While there is still a lack of accuracy associated with Chat GPT, we were both surprised by how much feedback it gave us after submitting a sentence and a hash.

Given the resources we had available to us, there is not much more we could have done to improve our project. We could have expanded our project to monitor multiple ports on multiple systems within a network if we had more time. We could have also designed our project to send an output to an output file instead of the terminal. That being said, our project ended up being closer to what we had originally envisioned than any one of our previous projects. Throughout most of our time at ISU, our projects have fallen short of our original goals. This time, we managed to (mostly) satisfy our original parameters for success, and that is something which we are both really happy about. I think it's safe to say that this project was a success!

Thank you for the great semester, Professor Sean Sanders!